4.1.1 Parser

The Parser checks the SQL statement and ensures its validity. The statements are verified both syntactically (the statement keywords must be properly spelled and following the SQL language guidelines) and semantically (the referenced tables and column do exist in the database).

During parsing, the SQL statement is transformed into a database-internal representation, called the *syntax tree* (also known as *parse tree* or *query tree*). If the SQL statement is a high-level representation (being more meaningful from a human perspective), the syntax tree is the logical representation of the database objects required for fulfilling the current statement.

4.1.2 Optimizer

For a given syntax tree, the database must decide the most efficient data fetching algorithm. Data is retrieved by following an *access path*, and the Optimizer needs to evaluate multiple data traversing options like:

- The access method for each referencing table (table scan or index scan).
- For index scans, it must decide which index is better suited for fetching this result set.
- For each joining relation (e.g. table, views or Common Table Expression), it must choose the best-performing join type (e.g. Nested Loops Joins, Hash Joins, Sort Merge Joins).
- The joining order becomes very important, especially for Nested Loops Joins.

The list of access path, chosen by the Optimizer, is assembled into an execution plan.

Because of a large number of possible action plan combinations, finding a good execution plan is not a trivial task. The more time is spent on finding the best possible execution plan, the higher the transaction response time will get, so the Optimizer has a fixed time budget for finding a reasonable plan.

The most common decision-making algorithm is the CBO (Cost-Based Optimizer). Each access method translates to a physical database operation, and its associated cost in resources can be estimated. The database stores various statistics like table sizes and data cardinality (how much the column values differ from one row to the other) to evaluate the cost of a given database operation. The cost is calculated based on the number of CPU cycles and I/O operations required for executing a given plan.

When finding an optimal execution plan, the Optimizer might evaluate multiple options, and, based on their overall cost, it chooses the one requiring the least amount of time to execute.

By now, it is clear that finding a proper execution plan is resource intensive, and, for this purpose, some database vendors offer execution plan caching (to eliminate the time spent on finding the optimal plan). While caching can speed up statement execution, it also incurs some additional challenges (making sure the plan is still optimal across multiple executions).

Each execution plan has a given memory footprint, and most database systems use a fixed-size cache (discarding the least used plans to make room for newer ones). DDL (Data Definition Language) statements might corrupt execution plans, making them obsolete, so the database must use a separate process for validating the existing execution plans relevancy.

However, the most challenging aspect of caching is to ensure that only a good execution plan goes in the cache, since a bad plan, getting reused over and over, can really hurt application performance.

4.1.2.1 Execution plan visualization

Database tuning would not be possible without knowing the actual execution plan employed by the database for any given SQL statement. Because the output may exceed the length of a page, some execution plan columns were removed for brevity sake.

Oracle

Oracle uses the `EXPLAIN PLAN FOR` syntax, and the output goes into the `dbms_xplan` package:

```
SQL> EXPLAIN PLAN FOR SELECT COUNT(*) FROM post;
SQL> SELECT plan_table_output FROM table(dbms_xplan.display());

-----------------------------------------------------------------
| Id  | Operation             | Name        | Rows  | Cost (%CPU)|
-----------------------------------------------------------------
|   0 | SELECT STATEMENT      |             |     1 |     5   (0)|
|   1 |  SORT AGGREGATE       |             |     1 |            |
|   2 |   INDEX FAST FULL SCAN| SYS_C007093 |  5000 |     5   (0)|
-----------------------------------------------------------------
```

PostgreSQL

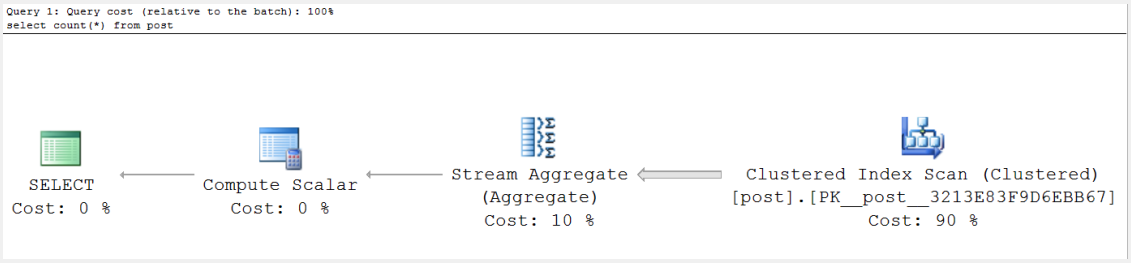
PostgreSQL reserves the `EXPLAIN` keyword for displaying execution plans:

```
EXPLAIN SELECT COUNT(*) FROM post;

QUERY PLAN
-------------------------------------------------------------
 Aggregate  (cost=99.50..99.51 rows=1 width=0)
   ->  Seq Scan on post  (cost=0.00..87.00 rows=5000 width=0)
```

SQL Server

The SQL Server Management Studio provides an execution plan viewer:



Another option is to enable the `SHOWPLAN_ALL` setting prior to running a statement:

```
SET SHOWPLAN_ALL ON;
GO
SELECT COUNT(*) FROM post;
GO
SET SHOWPLAN_ALL OFF;
GO

| Stmt Text                  | Est. Rows | Est. IO | Est. CPU | Subtree Cost |
-------------------------------------------------------------------------------
| select count(*) from post; | 1         | NULL    | NULL     | 0.0288       |
|   Compute Scalar           | 1         | 0       | 0.003    | 0.0288       |
|     Stream Aggregate       | 1         | 0       | 0.003    | 0.0288       |
|       Clustered Index Scan | 5000      | 0.020   | 0.005    | 0.0258       |
```

MySQL

The plan is displayed using `EXPLAIN` or `EXPLAIN EXTENDED`:

```
mysql> EXPLAIN EXTENDED SELECT COUNT(*) FROM post;

+----+--------+-------+-------+---------+-----+------+----------+-------------+
| id | select | table | type  | key     | key | rows | filtered | Extra       |
|    | type   | table | type  |         | len |      |          |             |
+----+--------+-------+-------+---------+-----+------+----------+-------------+
|  1 | SIMPLE | post  | index | PRIMARY | 8   | 5000 |   100.00 | Using index |
+----+--------+-------+-------+---------+-----+------+----------+-------------+
```

MySQL

When using MySQL 5.6.5[a] or later, you can make use of the `JSON EXPLAIN` format, which provides lots of information compared to the `TRADITIONAL EXPLAIN` format output.

```
mysql> EXPLAIN FORMAT=JSON select distinct title from post;
{
    "query_block": {
        "select_id": 1,
        "cost_info": {
            "query_cost": "1017.00"
        },
        "duplicates_removal": {
            "using_temporary_table": true,
            "using_filesort": false,
            "table": {
                "table_name": "post",
                "access_type": "ALL",
                "rows_examined_per_scan": 5000,
                "rows_produced_per_join": 5000,
                "filtered": "100.00",
                "cost_info": {
                    "read_cost": "17.00",
                    "eval_cost": "1000.00",
                    "prefix_cost": "1017.00",
                    "data_read_per_join": "3M"
                },
                "used_columns": [
                    "id",
                    "title"
                ]
            }
        }
    }
}
```

[a] https://dev.mysql.com/doc/refman/5.6/en/explain.html

4.1.3 Executor

From the Optimizer, the execution plan goes to the Executor where it is used to fetch the associated data and build the result set. The Executor makes use of the Storage Engine (for loading data according to the current execution plan) and the Transaction Engine (to enforce the current transaction data integrity guarantees).

Having a reasonably large in-memory buffer allows the database to reduce the I/O contention, therefore reducing transaction response time. The consistency model also has an impact on the overall transaction performance since locks may be acquired to ensure data integrity, and the more locking, the less the chance for parallel execution.

4.2 Caching performance gain

Before jumping into more details about *server-side* and *client-side* statement caching, it is better to visualize the net effect of reusing statements on the overall application performance. The following test calculates the number of queries a database engine can execute in a one-minute time span. To better emulate a non-trivial execution plan, the test executes a statement combining both table joining and query nesting.

```
SELECT p.title, pd.created_on
FROM   post p
LEFT JOIN post_details pd ON p.id = pd.id
WHERE  EXISTS (
    SELECT 1
    FROM   post_comment
    WHERE  post_id = p.id AND version = ?
)
```

Running it on four different database systems, the following throughput numbers are collected.

Table 4.1: Statement caching performance gain

Database System	No Caching Throughput (Statements Per Minute)	Caching Throughput (Statements Per Minute)	Percentage Gain
DB_A	419 833	507 286	20.83%
DB_B	194 837	303 100	55.56%
DB_C	116 708	166 443	42.61%
DB_D	15 522	15 550	0.18%

Most database systems can clearly benefit from reusing statements and, in some particular use cases, the performance gain is quite substantial.

Statement caching plays a very important role in optimizing high-performance OLTP (Online transaction processing) systems.

4.3 Server-side statement caching

Because statement parsing and the execution plan generation are resource intensive operations, some database providers offer an execution plan cache. The statement string value is used as input to a hashing function, and the resulting value becomes the execution plan cache entry key. If the statement string value changes from one execution to the other, the database cannot reuse an already generated execution plan. For this purpose, dynamic-generated JDBC `Statement(s)` are not suitable for reusing execution plans.

Forced Parameterization

Some database systems offer the possibility of intercepting SQL statements at runtime so that all value literals are replaced with bind variables. This way, the newly parameterized statement can reuse an already cached execution plan.

To enable this feature, each database system offers a vendor-specific syntax.

Oracle

```
ALTER SESSION SET cursor_sharing=force;
```

SQL Server

```
ALTER DATABASE high_performance_java_persistence SET PARAMETERIZATION FORCED;
```

Server-side prepared statements allow the data access logic to reuse the same execution plan for multiple executions. A `PreparedStatement` is always associated with a single SQL statement, and bind parameters are used to vary the runtime execution context. Because `PreparedStatement(s)` take the SQL query at creation time, the database can precompile the associated SQL statement prior to executing it.

During the precompilation phase, the database validates the SQL statement and parses it into a syntax tree. When it comes to executing the `PreparedStatement`, the driver sends the actual parameter values, and the database can jump to compiling and running the actual execution plan.

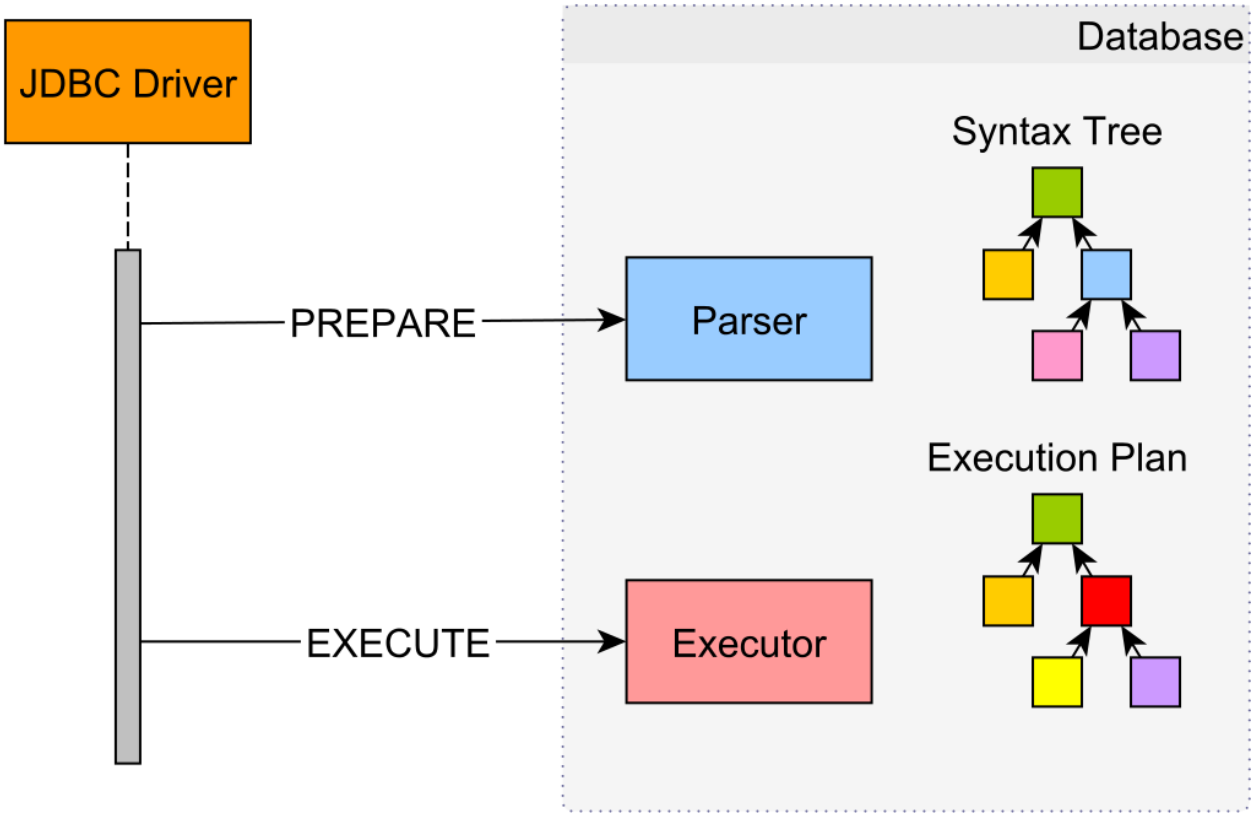


Figure 4.2: Server-Side prepared statement workflow

Conceptually, the prepare and the execution phases happen in separate database roundtrips. However, some database systems choose to optimize this process, therefore, multiplexing these two phases into a single database roundtrip.

Because of index selectivity, in the absence of the actual bind parameter values, the Optimizer cannot compile the syntax tree into an execution plan. Since a disk access is required for fetching every additional row-level data, indexing is suitable when selecting only a fraction of the whole table data. Most database systems take this decision based on the index selectivity of the current bind parameter values.

Because each disk access requires reading a whole block of data, accessing too many dispersed blocks can actually perform worse than scanning the whole table (random access is slower than sequential scans).

For prepared statements, the execution plan can either be compiled on every execution or it can be cached and reused. Recompiling the plan can generate the best data access paths for any given bind variable set while paying the price of additional database resources usage. Reusing a plan can spare database resources, but it might not be suitable for every parameter

value combination.

4.3.1 Bind-sensitive execution plans

Assuming a *task* table has a *status* column with three distinct values: *TO_DO*, *DONE*, and *FAILED*. The table has 100 000 rows, of which 1000 are *TO_DO* entries, 95 000 are *DONE*, and 4000 are *FAILED* records.

In database terminology, the number of rows returned by a given predicate is called cardinality and, for the *status* column, the cardinality varies from 1000 to 95 000.

$$C = \{1000, 4000, 95\ 000\}$$

By dividing cardinality with the total number of rows, the predicate selectivity is obtained:

$$S = \frac{C}{N} \times 100 = \{1\%, 4\%, 95\%\}$$

The lower the selectivity, the fewer rows are matched for a given bind value and the more selective the predicate gets. The Optimizer tends to prefer sequential scans over index lookups for high selectivity percentages, to reduce the total number of disk-access roundtrips (especially when data is scattered among multiple data blocks).

When searching for *DONE* entries, the Optimizer chooses a table scan access path (the estimated number of selected rows is 95 080):

```
SQL> EXPLAIN SELECT * FROM task WHERE status = 'DONE' LIMIT 100;

Limit  (cost=0.00..1.88 rows=100 width=13)
->  Seq Scan on task  (cost=0.00..1791.00 rows=95080 width=13)
     Filter: ((status)::text = 'DONE'::text)
```

Otherwise, the search for *TO_DO* or *FAILED* entries is done through an index lookup:

```
SQL> EXPLAIN SELECT * FROM task WHERE status = 'TO_DO' LIMIT 100;

Limit  (cost=0.29..4.25 rows=100 width=13)
->  Index Scan using task_status_idx on task  (cost=0.29..36.16 rows=907)
     Index Cond: ((status)::text = 'TO_DO'::text)

SQL> EXPLAIN SELECT * FROM task WHERE status = 'FAILED' LIMIT 100;

Limit  (cost=0.29..3.86 rows=100 width=13)
->  Index Scan using task_status_idx on task  (cost=0.29..143.52 rows=4013)
     Index Cond: ((status)::text = 'FAILED'::text)
```

So, the execution plan depends on bind parameter value selectivity. If the selectivity is constant across the whole bind value domain, the execution plan will no longer be sensitive to parameter values. A *generic* execution plan is much easier to reuse than a bind-sensitive one.

The following section describes how some well-known database systems implement server-side prepared statements in relation to their associated execution plans.

Oracle

Every SQL statement goes through the Parser, where it is validated both syntactically and semantically. Next, a hashing function takes the SQL statement, and the resulting hash key is used for searching the Shared Pool for an existing execution plan.

In Oracle terminology, reusing an execution plan is called a *soft parse*. To reuse a plan, the SQL statement must be identical with a previously processed one (even the case sensitivity and whitespaces are taken into consideration).

If no execution plan is found, the statement will undergo a *hard parse*[a]. The Optimizer evaluates multiple execution plans and chooses the one with the lowest associated cost, which is further compiled into a *source tree* by the *Row Source Generator*. Whether reused (*soft parse*) or generated (*hard parse*), the execution plan goes to the Executor, which fetches the associated result set.

Bind peeking

As previously mentioned, the Optimizer cannot determine an optimal access path in the absence of the actual bind values. For this reason, Oracle uses *bind peeking*[b] during the *hard parse* phase.

The first set of bind parameter values determines the selectivity of the cached execution plan. By now it is clear that this strategy is feasible for uniformly distributed data sets, and a single execution plan cannot perform consistently for bind-sensitive predicates.

As of 11g, Oracle has introduced adaptive cursor sharing so that a statement can utilize multiple execution plans. The execution plan is stored along with the selectivity metadata associated with the bind parameters used for generating this plan. An execution plan is reused only if its selectivity matches the one given by the current bind parameter values.

Both the execution plan cache and the adaptive cursor sharing are enabled by default, and, for highly concurrent OLTP systems, hard parsing should be avoided whenever possible. The plan cache allows database resources to be allocated to the execution part rather than being wasted on compiling, therefore improving response time.

`PreparedStatement(s)` optimize the execution plan cache-hit rate and are therefore preferred over plain JDBC `Statement(s)`.

[a] https://docs.oracle.com/database/121/TGSQL/tgsql_sqlproc.htm#TGSQL175
[b] https://docs.oracle.com/database/121/TGSQL/tgsql_cursor.htm#TGSQL848

SQL Server

SQL Server always caches execution plans[a] for both JDBC `Statement(s)` and `PreparedStatement(s)`. The execution plans are stored in the *procedure cache* region, and they are evicted only when the in-memory storage starts running out of space.

Even if SQL Server supports plain statements *forced parameterization*, preparing statements remains the most effective way to increase the likelihood of an execution plan cache-hit.

The catch is that all prepared statements should use the qualified object name, thus, the schema must always precede the table name.

So, instead of a query like this:

```
SELECT * FROM task WHERE status = ?;
```

the data access layer should always append the schema to all table names:

```
SELECT * FROM etl.task WHERE status = ?;
```

Without specifying the database object schema, the cache cannot determine which statistics to consider when analyzing the effectiveness of a given execution plan.

SQL Server inspects the actual parameter values during the first execution of a prepared statement. This process is called *parameter sniffing*, and its effectiveness is relative to predicate value distribution.

The database engine monitors statement execution times, and if the existing cached plan does not perform efficiently or if the underlying table structure or data distribution statistics undergo a conflicting change, then the database will recompile the execution plan according to the new parameter values.

For skewed data, reusing plans might be suboptimal, and recompiling plans on every execution could be a better alternative. To address the *parameter sniffing* limitations, SQL Server offers the `OPTION (RECOMPILE)` query hint[b], so the statement can bypass the cache and generate a fresh plan on every execution.

```
SELECT * FROM task WHERE status = ? OPTION(RECOMPILE);
```

[a] https://technet.microsoft.com/en-us/library/ms181055%28v=sql.100%29.aspx
[b] https://msdn.microsoft.com/en-us/library/ms181714.aspx

PostgreSQL

Prior to 9.2, a prepared statement was planned and compiled entirely during the prepare phase, so the execution plan was generated in the absence of the actual bind parameter values. Although meant to spare database resources, this strategy was very sensitive to skewed data. Since PostgreSQL 9.2, the prepare phase only parses and rewrites a statement, while the optimization and the planning phase are deferred until execution time. This way, the rewritten syntax tree is optimized according to the actual bind parameter values, and an optimal execution plan is generated.

For a single execution, a plain statement requires only a one database roundtrip while a prepared statement needs two (a prepare request and an execution call). To avoid the networking overhead, by default, JDBC `PreparedStatement(s)` do both the prepare and the execute phases over a single database request.

A client-side prepared statement must run at least 5 times for the driver to turn it into a server-side statement. The default execution count value is given by the `prepareThreshold` parameter, which is configurable as a connection property or through a driver-specific API[a].

After several executions, if the performance is not sensitive to bind parameter values, the Optimizer might choose to turn the plan into a generic one and cache it for reuse.

[a] https://jdbc.postgresql.org/documentation/publicapi/org/postgresql/PGStatement.html

MySQL

When preparing a statement, the MySQL Parser generates a syntax tree which is further validated and pre-optimized by a *resolution* mechanism. The syntax tree undergoes several data-insensitive transformations, and the final output is a *permanent tree*.

Since MySQL 5.7.4[a], all *permanent transformations* (rejoining orders or subquery optimizations) are done in the prepare phase, so the execution phase only applies data-sensitive transformations. MySQL does not cache execution plans, so every statement execution is optimized for the current bind parameter values, therefore avoiding data skew issues.

Because of some unresolved issues, since version 5.0.5[b], the MySQL JDBC driver only emulates server-side prepared statements. To switch to server-side prepared statements, both the `useServerPrepStmts` and the `cachePrepStmts` connection properties must be set to *true*.

Before activating this feature, it is better to check the latest Connector/J release notes and validate this feature is safe for use.

[a] http://mysqlserverteam.com/mysql-performance-schema-prepared-statements-instrumentation/
[b] https://dev.mysql.com/doc/relnotes/connector-j/5.1/en/news-5-0-5.html

4.4 Client-side statement caching

Not only the database side can benefit from caching statements, but also the JDBC driver can reuse already constructed statement objects. The main goals of the client-side statement caching can be summarized as follows:

- Reducing client-side statement processing, which, in turn, lowers transaction response time.
- Sparing application resources by recycling statement objects along with their associated database-specific metadata.

In high-performance OLTP applications, transactions tend to be very short, so even a minor response time reduction can make a difference in the overall transaction throughput.

Oracle implicit statement caching

Unlike server-side plan cache, the client one is confined to a database connection only. Since the SQL `String` becomes the cache entry key, `PreparedStatement(s)` and `CallableStatement(s)` have a better chance of getting reused. Therefore, the Oracle JDBC driver supports caching only for these two statement types. When enabling caching (disabled by default), the driver returns a logical statement, so when the client closes it, the logical statement goes back to the cache.

From a development point of view, there is an *implicit* statement caching mechanism as well as an *explicit* one. Both caching options share the same driver storage, which needs to be configured according to the current application requirements.

The implicit cache can only store statement *metadata*, which does not change from one execution to the other. Although it can be set for each individual `Connection`, it is convenient to configure it at the `DataSource` level (all connections inheriting the same caching properties):

```
connectionProperties.put("oracle.jdbc.implicitStatementCacheSize",
    Integer.toString(cacheSize));
dataSource.setConnectionProperties(connectionProperties);
```

Setting the `implicitStatementCacheSize` also enables the cache. By default, all executing statements are cached implicitly, and this might not be desirable (some occasional queries might evict other frequently executed statements). To control the statement caching policy, JDBC defines the `isPoolable()` and `setPoolable(boolean poolable)` `Statement` methods:

```
if (statement.isPoolable()) {
    statement.setPoolable(false);
}
```

Oracle explicit statement caching

The explicit cache is configurable and managed through an Oracle-specific API. Prior to using it, it must be enabled and resized using the underlying `OracleConnection` reference.

```
OracleConnection oracleConnection = (OracleConnection) connection;
oracleConnection.setExplicitCachingEnabled(true);
oracleConnection.setStatementCacheSize(cacheSize);
```

When using the explicit cache, the data access controls which statements are cacheable, so there is no need for using the `setPoolable(boolean poolable)` method anymore. The following example demonstrates how to make use of the explicit caching mechanism.

```
PreparedStatement statement = oracleConnection
    .getStatementWithKey(SELECT_POST_REVIEWS_KEY);
if (statement == null)
    statement = connection.prepareStatement(SELECT_POST_REVIEWS);
try {
    statement.setInt(1, 10);
    statement.execute();
} finally {
    ((OraclePreparedStatement) statement).closeWithKey(SELECT_POST_REVIEWS_KEY);
}
```

The explicit caching relies on two main operations, which can be summarized as follows:

1. The `getStatementWithKey(String key)` method loads a statement from the cache. If no entry is found, the `PreparedStatement` must be manually created using standard JDBC API.
2. The `closeWithKey(String key)` method pushes the statement back into the pool.

The vendor-specific API couples the data access code to the Oracle-specific API which hinders portability and requires a more complex data access logic (when accommodating multiple database systems).

Aside from caching *metadata*, the explicit cache also stores execution *state* and *data*. Although reusing more client-side constructs might improve performance even further, this strategy poses the risk of mixing previous and current execution contexts, so caution is advised.

SQL Server

Although the Microsoft SQL Server JDBC driver defines a `disableStatementPooling` property, as of writing (the 4.2 version), the statement cache cannot be enabled[a].

On the other hand, jTDS (the open source JDBC 3.0 implementation) offers statement caching on a per-connection basis. Being a JDBC 4.0-specific API, The `setPoolable(boolean poolable)` `Statement` method is not implemented in the 1.3.1 jTDS release. The cache has a default size of 500 entries which is also adjustable.

```
((JtdsDataSource) dataSource).setMaxStatements(cacheSize);
```

Even if jTDS has always focused on performance, the lack of a steady release schedule is a major drawback compared to the Microsoft driver.

[a] https://msdn.microsoft.com/en-us/library/ms378988%28v=sql.110%29.aspx

PostgreSQL

Since the PostgreSQL JDBC driver 9.4-1202[a] version, the client-side statements are cached, and their associated server-side statement keys are retained even after the initial `PreparedStatement(s)` is closed. As long as the current connection cache contains a given SQL statement, both the client-side `PreparedStatement` and the server-side object can be reused. The `setPoolable(boolean poolable)` method has no effect, and caching cannot be disabled on a per-statement basis.

The statement cache is controlled by the following connection properties:

- `preparedStatementCacheQueries` - the number of statements cached for each database connection. A value of 0 disables the cache, and server-side prepared statements are no longer available after the `PreparedStatement` is closed. The default value is 256.
- `preparedStatementCacheSizeMiB` - the statement cache has an upper memory bound, and the default value is 5 MB. A value of 0 disables the cache.

These properties can be set both as connection parameters[b] or as `DataSource` properties:

```
((PGSimpleDataSource) dataSource).setPreparedStatementCacheQueries(cacheSize);
((PGSimpleDataSource) dataSource).setPreparedStatementCacheSizeMiB(cacheSizeMb);
```

[a] https://jdbc.postgresql.org/documentation/changelog.html#version_9.4-1202
[b] https://jdbc.postgresql.org/documentation/head/connect.html#connection-parameters

MySQL

The statement caching is associated with a database connection, and it applies to all executing statements. In the 5.1.36 Connector/J driver version, the `setPoolable(boolean poolable)` method can disable caching for server-side statements only, the client-side ones being unaffected by this setting.

The client-side statement cache is configured using the following properties:

- `cachePrepStmts` - enables the client-side statement cache as well as the server-side statement validity checking. By default, the statement cache is disabled.
- `prepStmtCacheSize` - the number of statements cached for each database connection. The default cache size is 25.
- `prepStmtCacheSqlLimit` - the maximum length of a SQL statement allowed to be cached. The default maximum value is 256.

These properties can be set both as connection parameters[a] or at `DataSource` level:

```
((MysqlDataSource) dataSource).setCachePrepStmts(true);
((MysqlDataSource) dataSource).setPreparedStatementCacheSize(cacheSize);
((MysqlDataSource) dataSource).setPreparedStatementCacheSqlLimit(maxLength);
```

[a] http://dev.mysql.com/doc/connector-j/en/connector-j-reference-configuration-properties.html

5. ResultSet Fetching

Having discussed the SQL statement optimizations (batching and caching), it is time to move on to the response part of a query processing. Unlike the insert, update, and delete statements, which only return the affected row count, a JDBC select query returns a `ResultSet` instead.

The database Executor takes an execution plan and fetches data into a result set. Rows may be either extracted at once or upon being requested by the database client.

The SQL Standard defines both the result set and the cursor descriptor through the following properties:

- scrollability (the direction in which the result set can be iterated)
- sensitivity (when should data be fetched)
- updatability (available for cursors, it allows the client to modify records while traversing the result set)
- holdability (the result set scope in regard to a transaction lifecycle).

Following the standard specification, the JDBC `ResultSet` offers support for all the properties above.

Table 5.1: JDBC ResultSet properties

Property Name	Description
TYPE_FORWARD_ONLY	The result set can only be iterated from the first to the last element. This is the default *scrollability* value.
TYPE_SCROLL_INSENSITIVE	The result set takes a loading time snapshot which can be iterated both forward and backward.
TYPE_SCROLL_SENSITIVE	The result set is fetched on demand while being iterated without any direction restriction.
CONCUR_READ_ONLY	The result set is just a static data projection which does not allow row-level manipulation. This is the default *changeability* value.
CONCUR_UPDATABLE	The cursor position can be used to update or delete records, or even insert a new one.
CLOSE_CURSORS_AT_COMMIT	The result set is closed when the current transaction ends.
HOLD_CURSORS_OVER_COMMIT	The result set remains open even after the current transaction is committed.

5.1 ResultSet scrollability

The JDBC `ResultSet` can be traversed using an application-level cursor. The fetching mechanism is therefore hidden behind an iterator API, which decouples the application code from the data retrieval strategy. Some database drivers prefetch the whole result set on the client-side, while other implementations retrieve batches of data on a demand basis.

By default, the `ResultSet` uses a *forward-only* application-level cursor, which can be traversed only once, from the first position to last one. Although this is sufficient for most applications, JDBC also offers *scrollable* cursors, therefore allowing the row-level pointer to be positioned freely (in any direction and on every record).

The main difference between the two scrollable result sets lays in their *selectivity*. An *insensitive* cursor offers a static view of the current result set, so the data needs to be fetched entirely prior to being iterated. A *sensitive* cursor allows the result set to be fetched dynamically so it can reflect concurrent changes.

Oracle

Since the database engine does not offer support for scrollable result sets, the JDBC driver emulates it on top of a client-side caching mechanism[a]. As a consequence, the result set should not be too large as, otherwise, it can easily fill the client application memory.

A sensitive scrollable result set is limited to selecting data from a single table only.

[a] https://docs.oracle.com/database/121/JJDBC/resltset.htm#JJDBC28615

SQL Server

All three cursor types are supported. An insensitive scroll generates a server-side database snapshot, which the client fetches in batches. The sensitive scroll uses a server-side updatable window and changes are synchronized only for the current processing window.

The driver suggests using *read-only* cursors when there is no intent on updating the result set. The forward-only scroll delivers the best performance for small result sets[a].

PostgreSQL

By default, the result set is fetched entirely and cached on the client-side. Only the forward-only and the insensitive scroll are supported. For large result sets, fetching all records at once can put much pressure on both the database server resources and the client-side memory. For this purpose, PostgreSQL allows associating a result set to a database cursor[a] so records can be fetched on demand.

```
PreparedStatement statement = connection.prepareStatement(
    "SELECT title FROM post WHERE id BETWEEN ? AND ?"
);
statement.setFetchSize(100);
```

Only the forward-only result set type can benefit from database-side cursors, and the statement fetch size must be set to a positive integer value.

[a] https://jdbc.postgresql.org/documentation/head/query.html

MySQL

Only the insensitive scroll type is supported, even when explicitly specifying a forward-only result set. By default, the driver retrieves the whole result set and caches it on the client-side. Large result sets can be streamed[a] if the statement type is both forward-only and read-only and the fetch size value is set to `Integer.MIN_VALUE` value. This way, when iterating the `ResultSet`, each row will be fetched individually, which might incur multiple database roundtrips. Also, until the stream is closed, the connection cannot execute any other statement.

```
PreparedStatement statement = connection.prepareStatement(
    "SELECT title FROM post WHERE id BETWEEN ? AND ?"
);
statement.setFetchSize(Integer.MIN_VALUE);
```

Another option is to set the `useCursorFetch`[b] `Connection` property to `true` and then set the `Statement` fetch size to a positive integer value indicating the number of records that need to be fetched at a time.

```
statement.setFetchSize(100);
```

[a] http://dev.mysql.com/doc/connector-j/en/connector-j-reference-implementation-notes.html
[b] https://dev.mysql.com/doc/connector-j/8.0/en/connector-j-reference-implementation-notes.html

5.2 ResultSet changeability

By default, the result set is just a read-only view of the underlying data projection. Inspired by database cursors, the JDBC standard offers updatable result sets, so the data access logic can modify records while iterating the application-level cursor.

Mixing reading and writing logic into a single database transaction reminds of two-tier architectures, where holding the result set, even in the user think time, was both common and acceptable.

For web applications, requests should be as short as possible, and most application-level transactions span over multiple web requests. The former request may use a read-only database transaction to fetch data and render it to the user, while the latter might use a read-write transaction to apply data modifications. In such scenario, an updatable result set is of little use, especially because holding it open (along with the underlying database connection) over multiple requests can really hurt application scalability.

The following test case verifies if a forward-only and read-only cursor performs better than a sensitive and updatable one. The test executes 10 000 statements, fetching 100 posts along with *details* and their associated 1000 comments.

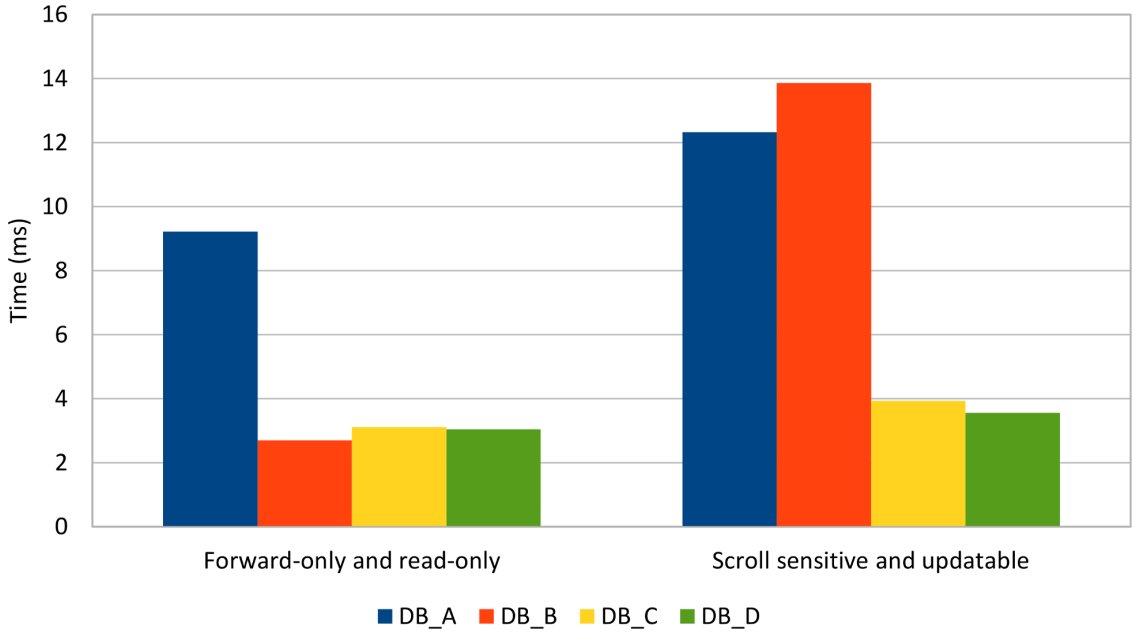


Figure 5.1: ResultSet cursor type

Every database system under test showed a slight improvement when using forward-only and read-only result sets.

As a rule of thumb, if the current transaction does not require updating selected records, the forward-only and read-only default result set type might be the most efficient option. Even if it is a reminiscence from JDBC 1.0, the default result set is still the right choice in most situations.

5.3 ResultSet holdability

The JDBC 3.0 version added support for result set holdability and, unlike scrollability and updatability, the default value is implementation-specific.

Oracle

The default and the only supported holdability value is `HOLD_CURSORS_OVER_COMMIT`. An exception is thrown when trying to change this setting to any other value.

SQL Server

By default, the result set is kept open even after the current transaction is committed or rolled back. SQL Server supports the `CLOSE_CURSORS_AT_COMMIT` setting as well.

PostgreSQL

Unlike other database systems, the default holdability value is `CLOSE_CURSORS_AT_COMMIT`, but the driver also supports the `HOLD_CURSORS_OVER_COMMIT` setting.

MySQL

The default and the only supported holdability value is `HOLD_CURSORS_OVER_COMMIT`.

In a typical enterprise application, database connections are reused from one transaction to another, so holding a result set after a transaction ends is risky. Depending on the underlying database system and on the cursor type, a result set might allocate system resources, which, for scalability reasons, need to be released as soon as possible.

Although the `CLOSE_CURSORS_AT_COMMIT` holdability option is not supported by all database engines, the same effect can be achieved by simply closing all acquired `ResultSet(s)` and their associated `Statement` objects.

5.4 Fetching size

The JDBC `ResultSet` acts as an application-level cursor, so whenever the statement is traversed, the result must be transferred from the database to the client. The transfer rate is controlled by the `Statement` fetch size.

```
statement.setFetchSize(fetchSize);
```

A custom fetch size gives the driver a hint as to the number of rows needed to be retrieved in a single database roundtrip. The default value of 0 leaves each database choose its driver-specific fetching policy.